

3 GIT HUB AND GIT

1. VERSION CONTROL SYSTEM :

- To maintain version in the repository we use version control
- A version control system (VCS) is the software that records changes made to source code, including separate files, sets of files, and any digital assets and related project metadata.
- This way, the system enables tracking of all project changes and allows developers to revert files to specific versions as needed. Without version control, projects risk becoming disorganized with numerous file versions.
- Also, VCS lets all team members work simultaneously on the latest file version for a specific project. This ensures faster and more efficient product development and delivery, crucial in the DevOps workflows. Moreover, VCS allows for conflict detection and resolution before issues reach production.
- VCS offer several benefits:
 - Version management: VCS stores all versions of changes down to the smallest developer commit.
 - Version restoration: Developers can restore specific versions to address conflicts or errors.
 - Security: VCS provides role-based access control, preventing unauthorized code access.
 - Integrity: Integration with other DevOps tools ensures stable processes.
 - Collaboration: VCS facilitates collaborative teamwork with a focus on individual work goals.
 - Backup: Modern VCS can serve as a backup for project codebases.

➔ There are two main types of version control systems:

- i. Centralized Version Control Systems (CVCS)
- ii. Distributed Version Control Systems (DVCS)

1.1 CVCS – CENTRALIZED VERSION CONTROL SYSTEMS :

- A centralized version control system operates on a client-server model. In this setup, the central server hosts the master repository, which contains all versions of the code. Developers begin by pulling the latest source code version to their local machines to make modifications. Once changes are made, they commit these changes to the central repository, where conflicts are resolved, and the updates are merged.

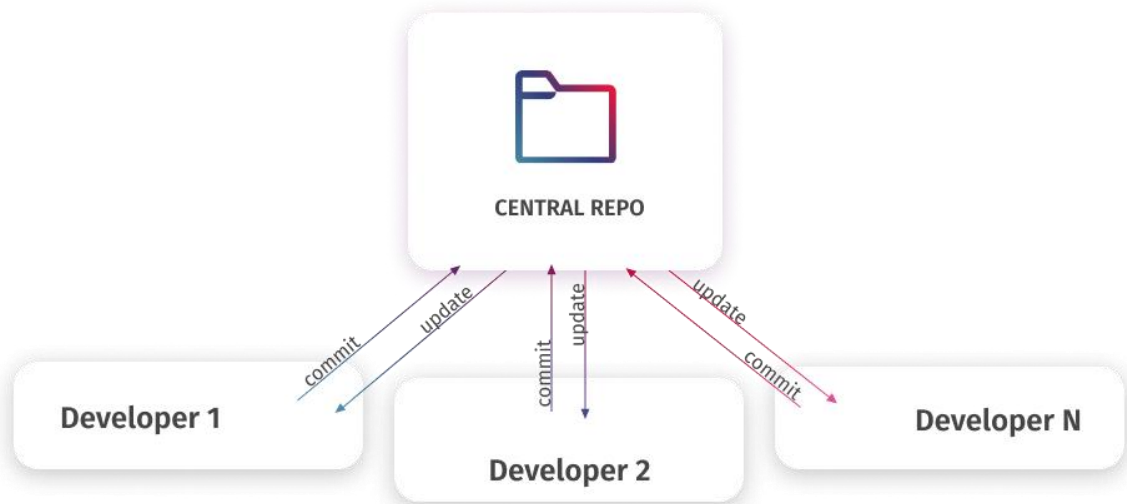
- Simple :

Centralized VCS model, only one developer can edit a file at a time. The file is locked during editing to prevent conflicts. Teams can create branches, but all changes go back to the central repository. This approach works best for small teams with good communication.

- Detailed :

In this model, only one developer can modify a specific piece of code at a time. The system locks the file to prevent other developers from accessing it while it is being edited. Team members may create branches to work independently, but all changes are eventually committed to the central repository. After merging, the server unlocks the files.

The centralized approach is best suited for smaller teams where direct communication and coordination are feasible, as this is vital for maintaining an effective workflow.



- CVCS = SVN

➔ **SVN – Subversion** is a version control system.

- It is used to track changes to files and folders over time.
- Helps teams collaborate on code, documents, or projects without losing previous versions.
- Subversion (SVN) is a widely used centralized version control system.
- Unlike other systems that support branching, SVN manages all project files in a single line.
- It simplifies scaling for large projects but requires comprehensive, robust security policies to control access to different areas of the project.

i. Key Features of SVN

- Version tracking – Keeps history of every change.
- Collaboration – Multiple people can work on the same project.
- Revert changes – Can undo mistakes by going back to previous versions.
- Branching and merging – Work on new features without affecting the main project.

ii. SVN Works : There is a central repository where all files are stored.

- Users check out files from the repository to their local machine.
- Make changes locally, then commit them back to the repository.

iii. Example workflow:

- svn checkout → get files from repository
- Edit files locally
- svn add → add new files
- svn commit → save changes to repository
- svn update → get latest changes from others

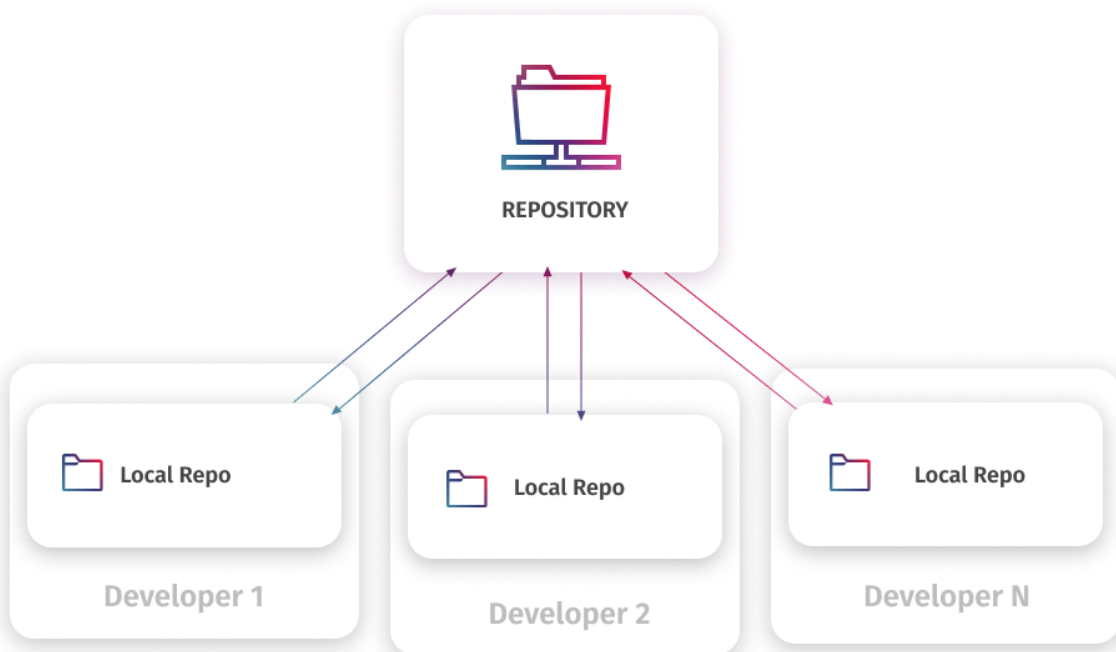
iv. Comparison with Git

Feature	SVN	Git
Repository type	Centralized	Distributed
History	Central server	Full local copy
Offline work	Limited	Fully offline

- SVN = Subversion, a centralized version control system to track changes and collaborate on files.

1.2 DVCS – DISTRIBUTED VERSION CONTROL SYSTEMS :

- DVCS function similarly to centralized models in most aspects but with a significant difference. Instead of a single server holding a repository, each developer maintains their own repository on the local machine.
That local repository contains the entire history and all branches.
In practice, using DVCS means that each user has a mirrored copy of the entire repository on their local machine. They can branch, commit, and merge changes locally without needing the server to store physical branch files; it only needs the commit differences.



- Simple :

In a distributed VCS, developers commit changes to their local repository first. They then push changes to the main repository after review. This allows independent work, offline progress, and ensures only high-quality code is merged.

- Detailed :

After making changes, the developer commits them to the local repository. However, at this stage, the local repository is separate from the master repository, resulting in different sets of changes between the developer's contribution and the master repository. Developers don't directly merge their code into the master repository. Instead, they request to push these changes from their local copy to the master repository.

The main advantage of the distributed model is that such systems allow users to work independently, even without a direct connection to the central repository. Therefore, even a failure in the central repository won't affect local work. Besides, with code review processes in place, only clean, high-quality code can be merged into the main repository.

➔ Centralized Version Control System Vs Distributed Version Control System

	Centralized	Distributed
Repository	Single central repository	Separate cloned repositories for every user
Connection	Requires constant connection to the central repository	Does not require constant connection, can work offline
Branching and merging	Limited	Extensive
History	The central repository contains the version history	Each local repository has a full version history
Access control	Permissions are set at the central server	Permissions are set at both local and remote repositories
Speed	Slower performance as operations rely on the central server	Faster performance as most tasks are done locally
Data loss risk	Failures on the central server can lead to data loss, and comprehensive backups are required	A full copy of the repository is available to each user, reducing data loss risks
Collaboration	Requires constant coordination with the central server	Allows concurrent work and less centralized control
Adaptability	More rigid	More adaptable

- Although DVCS can be tricky at first, it lets multiple developers work efficiently and deliver high-quality software.
- The most widely used examples of DVCS are Git and Mercurial.
- Git stands out as the most popular version control system overall. It's an open-source DVCS suitable for projects of any size and complexity, widely used in startups and enterprises alike.
- Mercurial is another DVCS with straightforward branching and merging features, ideal for scalable projects, and it offers an intuitive user interface. This visual mode makes it easy for even new users to quickly grasp the functionalities and work efficiently.

➔ Centralized vs Distributed

Centralized VCS workflow

Distributed VCS workflow

Users connect to the central repository where all project files are stored and download the latest version to their local machines, also updating their local copies to minimize conflicts.	Every developer has a local clone of the entire repository, including all branches and changes history. Work starts by updating this local copy with the latest changes from the central repository.
Users make changes to the files on their computers.	Developers work locally, creating work branches, making commits, and merging local branches with the master/main branch. These operations are possible even offline.
Once all local changes are finalized, users commit them to the central repository, making their changes accessible to all other users.	After committing and merging local branches, developers push changes to the remote master repository, making all their changes available to others.

- The choice between these version control systems depends on each team's specific goals and environment.

➔ Advantages and disadvantages of CVCS vs. DVCS

- Centralized VCS:

Advantages or Pros:	Disadvantages or Cons :
High performance for binary files	Single point of failure: If the centralized server goes down, work halts.
Complete visibility of code for all team members	Slower speed due to constant communication with the remote server
The lower learning curve for system setup and workflow implementation	Less flexibility in customizing workflows

- Distributed VCS:

Advantages or Pros:	Disadvantages or Cons:
Excellent branching and merging capabilities	Steep learning curve requiring advanced technical skills
Management of offline changes	Higher disk space consumption as each clone contains full project history
Redundancy and backup features	Complex conflict management in heavily branched projects
High flexibility supporting diverse workflows	
Better performance in distributed mode	

Sir explain :

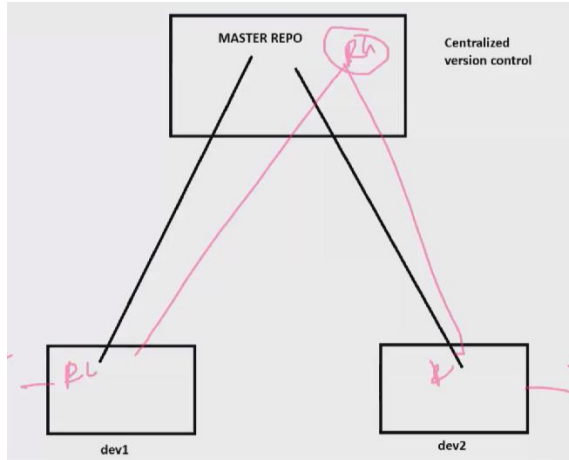
- i. Centralized version control : SVN
- ii. Distributed version control : Git, Github

- i. CVCS – Centralized Version Control System

Works in server and client control

Master code will be in server

Client will download code and develop



In master repo we have rg

Dev 1 will pull or download rg and develop vnet sub

Dev 2 will pull and develop vm, lb and firewall

1st download and upload

In centralized once master repo is down due to some issue

It is single point failure

We need to wait until it comes back because we can not pull push

But in distributed control also works as a server and client concept

In distributed when we pull code

In local pc local server gets created and code executed will be in local server

But when system gets crash code gets lose or in company many use same code

Hence it is used as distributed

As code saved in master or main repo

At any point of time updated code is in local

When dev 2 have uploaded or pushed some code

Them dev 1 need to pull code else code will not push

Always master code will be in local repo

2. GIT INTRODUCTION

➔ Git is a version control system.

- Git helps you track changes in your code or files over time.
- Software application need to install local
- You write code today, Tomorrow you change it and something breaks, You want the old working version back
- Git solves this by keeping a **history of your project**.

➔ Important Git concepts (simple)

Term	Meaning
Repository (repo)	A folder that Git tracks
Commit	A saved version of your work
Branch	A separate copy to work on new ideas
Main/Master	The main working branch
Clone	Copy a repo to your computer
Merge	Combine changes

➔ What Git works on

- Your computer
- No internet required
- Controlled using the terminal/command line

3. GIT HUB INTRO

➔ GitHub is a website that stores Git repositories online.

- If Git is the tool, GitHub is the place 🏠.
- Centralized remote registry

➔ Why GitHub is needed

I. Git alone: Works on your computer, Good for solo work

II. GitHub : Stores your code on the internet,

- Lets you share your code
- Helps people work together
- Acts as a backup
- You need to have account in Github

➔ Simple

- Git = saving progress on your PC
- GitHub = saving progress in the cloud ☁

➔ What GitHub allows you to do

- Upload your Git projects
- Download other people's projects
- Work with friends or teams

- Show your projects to teachers or companies
- Track bugs and tasks

Important GitHub concepts

Term	Meaning
Remote repo	Repo stored on GitHub
Push	Send your code to GitHub
Pull	Get latest code from GitHub
Fork	Copy someone's repo to your account
Pull Request (PR)	Request to merge your changes

➔ How Git and GitHub work together : Step-by-step flow (simple)

- Create a project on your computer
- Git tracks the project
- You make changes
- Git saves them (commit)
- You send them to GitHub (push)
- Others can download your code (pull)

➔ Very simple diagram

- Your Computer (Git) <----> GitHub (Online)

➔ Real-life analogy : Notebook example 📓

- Git = writing pages and keeping old pages
- Commit = saving a page
- Branch = starting a new notebook page for ideas
- GitHub = photocopying the notebook and sharing it with others

➔ Why students & developers use GitHub

- Project submission
- Team projects
- Open-source learning
- Portfolio for jobs
- Safe backup of work

➔ Git Vs GitHub

Git	GitHub
Tool/software	Website/platform
Works offline	Needs internet
Tracks changes	Stores & shares projects
Version control	Collaboration

Feature	Git	GitHub
Type	Tool / Software	Platform / Website
Works offline	☒ Yes	☒ No
Purpose	Track code changes	Share & collaborate
Requires Git	—	☒ Yes

Need	Version Control	Collaboration
------	-----------------	---------------

4. GIT AREAS

- A Git area refers to the different stages (areas) a file goes through while being tracked by Git.
- There are three main Git areas.
 - i. Working Directory or Area
 - ii. Staging or Indexing Area
 - iii. Repository or Commit Area

I. Working Directory (Working Area)

- This is the folder on your computer
- Where you write and edit your files
- This is where you are currently working 🍷
- Git knows the file changed, but it's not ready to be saved.

II. Staging Area (Index)

- A middle area between working directory and repository
- Where you choose what changes to save
- This is like a waiting room 🚪
- You decide which changes should be included in the next save.

• Example:

- You changed 3 files
- You only want to save 2 now
- You add those 2 files to the staging area

- This gives you control over what goes into a commit.

III. Repository (Local Repository)

- Where Git permanently saves your project history
- Contains all commits
- This is the safe storage 🗄️
- Once changes reach here, they are officially saved.

➔ Simple Flow Diagram

- Working Directory
 - ↓
- Staging Area
 - ↓
- Repository

➔ Git Commands for Each Area

Area	Common Command
------	----------------

Working Directory	edit files
Staging Area	git add file_name
Repository	git commit -m "message"

➔ Bonus: Remote Repository (Extra)

IV. Remote Repository (GitHub)

- Stored online
- Used for sharing and backup
- Commands:
 - git push → send code to GitHub
 - git pull → get code from GitHub

➔ Simple

- Working Directory → You edit files
- Staging Area → You prepare files
- Repository → Git saves files / Save History
- Remote Repository → Online storage / Share Online

5. GIT INSTALLATION

- Download git software
 - Open chrome and type git download
 - Open 1st link of git official account
 - <https://git-scm.com/install/windows>
 - Click on git 64 bit download or standalone installer 64 bit
 - Or click on download option

git --distributed-even-if-your-workflow-isnt

Search: Type / to search entire site...

About
Learn
Tools
Reference
Install
Community

Install

Latest version: 2.52.0 ([Release Notes](#))

Windows | macOS | Linux | Build from Source

Click here to download the latest (2.52.0) x64 version of **Git for Windows**. This is the most recent **maintained build**. It was released **39 days ago**, on 2025-11-17.

Other Git for Windows downloads

Standalone Installer
Git for Windows/x64 Setup.
Git for Windows/ARM64 Setup.
Portable ("thumbdrive edition")
Git for Windows/x64 Portable.
Git for Windows/ARM64 Portable.

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to **read online for free**. Dead tree versions are available on [Amazon.com](#).

➔ Git installation

- Once github is installed to run or execute codes
- We can execute using cmd, powershell and gitbash
- Gitbash has default linux

➔ After install

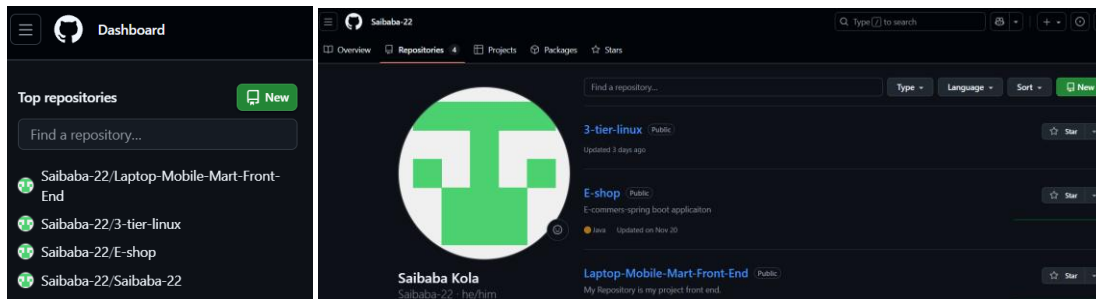
- After installing, We can observe in start button git bash, github and git lab

ii. From github

- Create an account
- If already have login

➔ From Github

- Create repository :-
- Top left you can find new option



Or

- From my profile -> profile -> repository -> new
- Click on it
 - i. Under general :
 - Owner : account holder name
 - Repository name : foldername

The image shows the 'Create a new repository' form on GitHub. It includes a 'General' tab with fields for 'Owner' (set to 'Saibaba-22'), 'Repository name', and 'Description'. A note suggests repository names should be short and memorable, with an example 'expert-octo-tribble?'. The character count for the description is shown as '0 / 350 characters'.

ii. Configuration

- Choose visibility :
 - Public : any one or everyone can access it
 - Private : only person with link can access
- Click on create repository
- Repository have created

6. GIT PAT – PERSONAL ACCESS TOKEN

➔ Git PAT – Personal Access Token

- A PAT is a secure, random string that is used instead of a password to authenticate (log in) to Git hosting services like GitHub, GitLab, Azure DevOps, Bitbucket, etc.

➔ Create PAT token process

- From git hub -> Git profile Icon -> setting -> at left option scroll bottom you can find developer settings -> PAT -> Classic token
- Fine – grained token
- Classic token
- Click on classic -> generate new token -> new token classic
- Provide password to get it
- Provide all permissions
- Click on generate token

➔ Provide pat in git

- For windows and git
 - For windows we use pat key for cloning
 - When you try to push files to Github it would ask git token or credentials
 - Provide it and done
- For Linux
 - We use SSH-keygen
 - Now copy public key save it and enter
 - Cat <key> (Copy the key)
 - Github settings -> deploy keys -> add deploy key
 - Title : any thing
 - Key : paste the key
 - Add key
 - Now you can add files from linux also
 - Git has default value as name and folder
 - Enter git config --global --edit
 - Edit name and email (for real time)

Once any user or developer uploads or pushes any code
1st git pull then you can push

Git platforms (especially GitHub) offer two types of Personal Access Tokens (PATs):

- i. Classic tokens

ii. Fine-grained tokens.

I. Classic Token (Classic PAT) : A Classic token is the old / traditional type of Personal Access Token.

➔ How it works

- Permissions are given using broad scopes
- Access applies to all repositories you have permission to

➔ Characteristics

- ✗ Too much access
- ✗ Less secure
- ✗ Cannot limit access to a single repo
- ☑ Still widely supported by tools

➔ Use case

- Legacy tools
- Older CI/CD pipelines
- When fine-grained tokens are not supported

II. Fine-Grained Token : A Fine-grained token is a newer, more secure type of GitHub PAT.

➔ How it works

- Permissions are very specific
- Access can be limited to : Single repository
 - Specific actions (read/write)
- Requires approval in some orgs

➔ Example permissions

Read issues only

Write contents to one repo

Read pull requests

➔ Characteristics

- ☑ Least-privilege security
- ☑ Repository-specific
- ☑ Better auditing
- ✗ Not supported by all tools yet

➔ Use case : New projects

- CI/CD pipelines
- Automation with high security requirements

➔ Comparison

Feature	Classic Token	Fine-Grained Token
Access scope	All repos	Specific repos

Permission control	Broad	Very specific
Security	Lower	Higher
Expiry required	Optional	Mandatory
Organization approval	No	Sometimes
Recommended	☒ No	☐ Yes

➔ Simple :

- Classic token → Master key to the whole building
- Fine-grained token → Key to one room only

Classic tokens provide broad access to all repositories using scopes.

Fine-grained tokens offer repository-specific and permission-specific access, making them more secure.

7. GIT BASIC COMMANDS

➔ Create or open a folder where you want to start git project

- Open folder where you want to execute commands and right click and click open git bash

Check hidden folder if git is present or not

If present git project else create it by executing

- To start a git project : git init
Without git supporting files git does not work
- For creating files using git : touch <filename with extension code>
Touch file.txt
- To move from working area to index area or track : git add
- For tracking files single : git add <filename>
- For tracking multiple files : git add .
- Use cd folder name to change folder or go into folder : cd <foldername>
cd : change directory
- Use to list all the files : ls
- To check status of files : git status
- To move from staging to commit area : git commit -m "som msg"
- To check any commits or files present in commit area : git log
It shows who is auther, when it is done and commit id
- To provide commit id in 6-7 char use : git log --oneline
- ➔ For file name color
 - When we create any files we will be in working area as 1st area as marked as red
 - When we use git status it shows untracked file (git does not know) what file it is
 - Using git add to move from working area to staging area and marked as green
 - Use git commit -m "some" to move from staging area to commit area

➔ To upload to git and clone from git

- To upload from git local pc to git repo : Git push u origin main
- To download or clone from git repo : git clone <gitrepo-url>

For download or cloning complete files and folders present in it
From master to local repo

- To move from committing area to github : git push

Before : git push

Quick setup — if you've done this kind of thing before

 Set up in Desktop
 or
 HTTPS
 SSH
 <https://github.com/Saibaba-22/git-trail.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include

...or create a new repository on the command line

```

echo "# git-trail" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Saibaba-22/git-trail.git
git push -u origin main

```

After git push

```

$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 478 bytes | 239.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Saibaba-22/git-trail.git
 * [new branch]      main -> main

```


git-trail
Public

 Pin
  Watch

 main
  1 Branch
  0 Tags



 Add file
  Code

 Saibaba-22	file2	fbfaa86 · 5 minutes ago	 2 Commits
 file2	file2	5 minutes ago	
 trail.txt	move	14 minutes ago	

- Now modify trail with some content
- Now trail will be in untracked due to modification
- For tracking multiple files : git add .
- To move from staging to commit area : git commit -m "som msg"
- When we check logs it will show us 3 files
 - i. Trail old file and new file name
 - ii. One more file
- When we add to github there will be only 2 files as
 - i. One file created and one modified

Saibaba-22 vnm		4aa8140 · 2 minutes ago	🕒 3 Commits
📄 file2	file2	14 minutes ago	
📄 trail.txt	vnm	2 minutes ago	

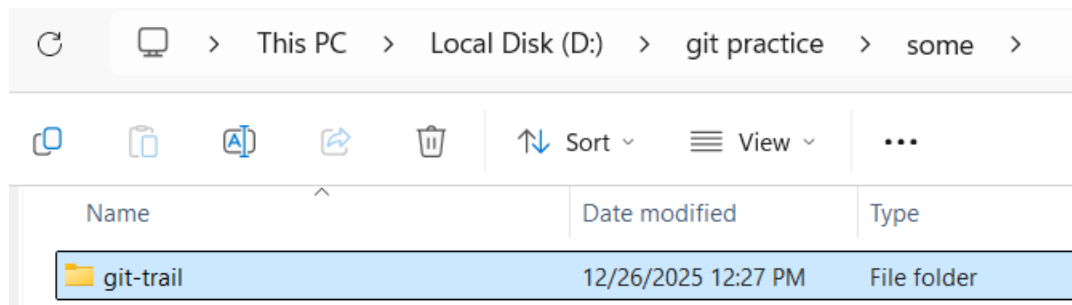
- It will show timing also when It is modified

Git commands

- Use this command for come out of folder : cd ..
- To create a new folder use this : mkdir <foldername>
- Use this to move to created folder : cd <foldername>/
- ➔ To clone complete files and folders from github using github it
 - Open web and open github profile -> git repository
 - Click on code and under local you can find git link or Clone using the web URL copy it
 - Use git clone <gitlink>
 - For download or import complete data into local pc

```
HOME@DESKTOP-L5ACAPJ MINGW64 /d/git practice/some
$ git clone https://github.com/Saibaba-22/git-trail.git
Cloning into 'git-trail'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 9 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
```

- Now you can check in local pc as complete git repo is downloaded or cloned with repo name



8. GIT BRANCHING STRATEGY

➔ A Git branching strategy is a structured way a team uses branches in Git to develop, test, and release code safely and efficiently.

- To isolate from one code to another we need branches
- It defines which branches exist, what they're for, and how/when they're merged.

➔ Branching strategies matter : They help teams in

- Work in parallel without breaking code
- Keep the main codebase stable
- Release features and fixes in a controlled way
- Collaborate without conflicts

I. Git Flow (Classic, Structured)

- Best for: Large projects, scheduled releases, multiple developers

➔ Main branches

- main (or master) → production-ready code
- develop → integration branch for features

➔ Supporting branches

- feature/* → new features (from develop)
- release/* → prepare a release
- hotfix/* → urgent production fixes (from main)

➔ Flow : feature → develop → release → main ↘ develop

➔ Pro and Cons

Pros	Cons
Very organized	Heavy process
Clear separation of concerns	Slower for fast-moving teams

II. GitHub Flow (Simple & Popular)

- Best for: Web apps, CI/CD, small to medium teams

➔ Branches

- main → always deployable

- feature branches → short-lived

➔ Flow : main → feature → Pull Request → main

➔ Rules : Create a branch for each feature or fix

- Open a pull request
- Merge after review and tests pass

➔ Pros : Simple, Fast, Works well with automation

➔ Cons : Less structure for long release cycles

III. GitLab Flow (Hybrid)

- Best for: Teams needing environments (staging, production)

➔ Branches

- main
- feature/*
- environment branches (e.g., staging, production)

➔ Flow : feature → main → staging → production

➔ Pros : Environment-focused, Clear deployment stages

➔ Cons : Slightly more complex than GitHub Flow

IV. Trunk-Based Development

- Best for: High-performing teams, frequent releases

➔ Branches

- main (trunk) → everyone commits frequently
- Very short-lived feature branches (hours/days)

➔ Key ideas : Small commits, Feature flags, Strong automated testing

➔ Pros : Very fast delivery, Fewer merge conflicts

➔ Cons : Requires discipline and good tests

V. Release Branching (Enterprise Style)

- Best for: Long-term support versions

➔ Branches

- main
- release/x.y (e.g., release/1.2)
- hotfix/*

➔ Pros : Supports multiple versions in parallel

➔ Cons : More maintenance effort

❖ Simple

Team / Project Type	Recommended Strategy
Solo / small team	GitHub Flow
CI/CD web app	GitHub Flow
Large enterprise	Git Flow
Fast, mature team	Trunk-Based
Multiple live versions	Release Branching

9. MERGE CONFLICTS

- Create a repo in github and copy url
- To add the github repository to git : `git remote add <name> <url>`
- In github repo master is a branch and we can create multiple branches
- To push code to main branch of git repo : `Git push <name> master`

`git remote add <name> <url>`

ex : `git remote add repo1 <url>`

in this way we can add multiple repo as we need to push code or send to destinations

- To add a new branch : `git branch <branchname>`
- To check in which branch you are : `git branch`
- In git bash * mark indicates current branch
- To switch from one branch to another : `git switch "branchname"`
- To delete branch : `git branch -d <branchname>`
- To create a new branch and copy the entire files to new branch and switch to new branch : `git checkout -b <branchname>`
So that changes made will not effect old branch and It is safe
- To push code to another branch which is created : `git push origin feature`
`git push <origin git> <branchname>`
- From git hub : compare & pull request
 - Select from and to branches in github it shows reverse make it as it shows
 - Add title write some name
 - Add description some name
 - Create pull request
 - If there are another editor or owner it shows owner at right side
 - Click on merge pull request
 - Confirm merge
 - From second branch code transfers to 1st branch

10. PR OR PULL REQUEST

➔ A Pull Request is a request to pull your code changes from one branch into another (for example, feature/login → main).

- PR request and Pull Request usually mean the same thing, but the wording can be confusing.
- A Pull Request is the correct and standard term.

➔ Pull request work flow

- You create a feature branch
- You push commits to it
- You open a Pull Request
- Teammates review the code
- Automated tests run
- The PR is approved and merged

➔ Used by

- GitHub → Pull Request
- GitLab → Merge Request (same concept)
- Bitbucket → Pull Request

“PR request” literally means “Pull Request request”

Pull request for Correct and formal

“PR request” is informal and redundant.

➔ Correct alternatives

- “Create a PR”
- “Open a pull request”
- “Send a PR”
- “Review my PR”

➔ Example Workflow

git checkout -b feature/add-login

git commit -m "Add login feature"

git push origin feature/add-login

Then on GitHub:

Click New Pull Request

Request reviewers

Merge after approval

➔ Simple :

Term	Correct?	Meaning
Pull Request	☑ Yes	Official Git term
PR	☑ Yes	Short for Pull Request
PR request	✗ No	Redundant / informal

11. HOT FIX BRANCH

➔ A hotfix branch is a special Git branch used to quickly fix a critical bug in production.

- A hotfix branch is created when:
 - A serious bug is found in the live (production) system

- The fix can't wait for the next normal release cycle
- It lets you patch production without disturbing ongoing feature work.

➔ Where It Fits (Git Flow) : In Git Flow, branches usually look like this:

- main → production code
- develop → ongoing development
- feature/* → new features
- hotfix/* → emergency production fixes

➔ How a Hotfix Branch Works

1. Create hotfix from main

```
git checkout main
git pull
git checkout -b hotfix/fix-payment-crash
```

2. Fix the bug & commit

```
git commit -am "Fix payment crash in production"
```

3. Merge back into main

```
git checkout main
git merge hotfix/fix-payment-crash
```

4. Also merge into develop

```
git checkout develop
git merge hotfix/fix-payment-crash
```

- This ensures the fix is in production AND future releases.

➔ Key Characteristics

- 🚨 Urgent
- ⌚ Short-lived
- 🔒 Based on production code
- 🔗 Merged back into all active branches

➔ When to Use a Hotfix Branch

- Use when:
 - App is down
 - Security vulnerability
 - Data loss or corruption
 - Payment / login failure in production
- Don't use when:
 - It's a normal bug
 - It can wait for the next release

➔ Naming Convention Examples

hotfix/login-500-error

hotfix/security-patch

hotfix/crash-on-startup

➔ Hotfix vs Feature vs Bugfix

Branch Type	Purpose	Urgency
Feature	New functionality	Low
Bugfix	Non-critical bug	Medium
Hotfix	Production emergency	High

➔ A hotfix branch is a fast emergency lane to fix production bugs without waiting for a full release.

12. GIT ADD NEW TOKEN AND REMOVE OLD

➔ To remove old credentials : `git config --global --unset credential.helper`

➔ For clearing cached credentials : `rm -rf ~/.git-credentials`

➔ L

`git pull <remote> <branch>`

13. GIT TOTAL COMMANDS

(Explained by Sir)

➔ Basic Git Commands (Most Important)

`git init` : Creates a new Git repository.

`git status` : Shows the current state of files (changed, staged, untracked).

`git add` : Adds files to the staging area.

For single file with file name : `git add file.txt`

For all files : `git add .`

`git commit` : Saves staged changes permanently.

`git commit -m "Added homepage design"`

`git log` : Shows commit history.

`git diff` : Shows changes between versions.

➔ Working with Files

`git rm` : Deletes a file and tracks the deletion.

`git rm file.txt`

`git mv` : Renames or moves a file.

`git mv old.txt new.txt`

➔ Branching Commands

9. `git branch` : Shows or creates branches.

`git branch`

`git branch new-feature`

10. git checkout : Switches branches or restores files.

git checkout main

git checkout new-feature

11. git switch (newer & simpler) : Switches branches.

git switch main

12. git merge : Combines branches.

git merge new-feature

Remote (GitHub) Commands

13. git clone : Copies a repository from GitHub.

git clone <https://github.com/user/repo.git>

14. git remote : Manages remote repositories.

git remote -v

15. git push : Uploads commits to GitHub.

git push origin main

16. git pull : Downloads and updates code from GitHub.

git pull origin main

17. git fetch : Downloads updates without merging.

git fetch

Undo & Fix Commands

18. git reset : Removes files from staging or moves commits back.

git reset file.txt

19. git restore : Restores files to last committed version.

git restore file.txt

20. git revert : Creates a new commit that undoes a previous commit.

git revert commit_id

21. git stash : Temporarily saves unfinished work.

git stash

git stash pop

Info & Help Commands

22. git show : Shows details of a commit.

git show commit_id

23. git config : Sets user name and email.

git config --global user.name "Your Name"

git config --global user.email "you@email.com"

24. git help : Shows help for Git commands.

git help commit

◆ Quick Summary Table

Command	Purpose
git init	Start Git
git status	Check file status
git add	Stage files
git commit	Save changes
git log	See history
git branch	Manage branches
git checkout/switch	Change branch
git merge	Combine branches
git clone	Copy repo
git push	Upload code
git pull	Download code
git stash	Save work temporarily

pick → keep commit

reword → change message

edit → change commit

squash → combine commits

drop → remove commit

git squash Combine commits

git stash Temporarily save changes

git rebase Reapply commits on new base

git backup

git migration

git snapshot

➔ If there are codes in online and need to push code to git hub

- Now enter : git clone <remoterepourl>

We can not push code to github directly

- Commands : git clone <remoteurl> for pulling

Cd /** command to change folder or directory

- For creating file : Cat > file.txt
description

BASIC GIT COMMANDS

- `git init` : Initialize a new Git repository
- `git clone` : Copy a remote repository
Get project locally

`git clone <url>`

- `git status` : Shows repo state
Use: Check staged/unstaged files
- `git add` : Stage files
Use: Prepare for commit

`git add .`

- `git commit` : Save changes
Use: Create snapshot

`git commit -m "msg"`

- `git log` : View commit history
Use: Inspect commits
- `git log --oneline` : View commit history in 5 to 7 characters
Use : Inspect commits short
- `git diff` : Show changes
Use: Review code differences
- `git show` : Show commit details
Use: Inspect a commit

- `git config` : Configure Git
Use: Set username/email

`git config --global user.name "Name"`

- `git help` : Git documentation
Use: Learn commands

● INTERMEDIATE GIT COMMANDS (DAY-TO-DAY)

- `git branch` : Manage branches
Use: Create/list/delete branches
- `git checkout` : Switch branches/files
Use: Move between branches
- `git switch` : Switch branches (modern)
Use: Safer than checkout
- `git merge` : Combine branches
Use: Integrate feature branch
- `git pull` : Fetch + merge
Use: Update local branch
- `git push` : Upload commits
Use: Share work
- `git fetch` : Download updates only
Use: Review before merge
- `git remote` : Manage remotes
Use: Add/remove origin
- `git stash` : Save uncommitted changes
Use: Switch tasks quickly
- `git stash pop` : Apply + delete stash
Use: Restore work
- `git restore` : Restore files
Use: Undo file changes
- `git rm` : Delete tracked files
Use: Remove files properly
- `git mv` : Rename/move files
Use: Track renames

● ADVANCED GIT COMMANDS (POWER TOOLS)

- `git reset` : Move HEAD
Use: Undo commits

- `git revert` : Undo safely
Use: Shared branches
- `git rebase` : Reapply commits
Use: Clean history
- `git rebase -i` : Interactive rebase
Use: Edit/squash commits
- `git cherry-pick` : Pick specific commits
Use: Apply selected fixes
- `git tag` : Mark releases
Use: Versioning
- `git blame` : Line-by-line author
Use: Debugging
- `git reflog` : HEAD history
Use: Recover lost commits
- `git clean` : Remove untracked files
Use: Cleanup repo
- `git shortlog` : Summary of commits
Use: Contributor stats
- `git describe` : Describe commit
Use: Release naming
- `git archive` : Create source archive
Use: Export code

● EXPERT / RARELY USED COMMANDS

- `git bisect` : Find bug commit
Use: Debug regressions
- `git submodule` : Manage sub-repos
Use: Include other repos
- `git subtree` : Embed repo
Use: Monorepos
- `git worktree` : Multiple working trees
Use: Parallel branches

- `git bundle` : Package repo
Use: Offline sharing
- `git fsck` : Repo integrity check
Use: Repair corruption
- `git gc` : Garbage collect
Use: Optimize repo
- `git count-objects` : Object stats
Use: Repo size analysis
- `git rev-parse` : Parse refs
Use: Scripting
- `git verify-commit` : Verify commit signatures
Use: Security
- `git notes` : Attach notes
Use: Metadata
- `git grep` : Search repo
Use: Fast code search
- `git range-diff` : Compare commit ranges
Use: Review rebases
- `git merge-base` : Find common ancestor
Use: Advanced merges
- `git show-ref` : Show refs
Use: Debug refs

CONFIG & MAINTENANCE COMMANDS

- `git config --list` : Show config
Use: Debug settings
- `git config --unset` : Remove config
Use: Reset settings
- `git prune` : Remove unreachable objects
Use: Cleanup
- `git update-ref` : Update refs manually
Use: Advanced scripting

- `git symbolic-ref` : Manage symbolic refs
Use: Ref control
- `git daemon` : Git server
Use: Hosting repos
- `git instaweb` : Repo browser
Use: Visual history
- `git whatchanged` : Commit diff summary
Use: History review
- `git commit --amend` : Modify last commit
Use: Fix mistakes
- `git diff --staged` : Diff staged files
Use: Review before commit